

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 7
Operating Systems
Topic: Processes and the System-Call Interface (Linux)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 INSPECTING REAL SHELL SYSTEM CALLS **4**

2.1 System Call Tracing and Execution 4

2.2 Analysis and Observations 4

3 PROCESS LIFECYCLES, ZOMBIES, AND BUFFERING **6**

3.1 Implementations: `fork.c`, `zombie.c`, and `quiz.c` 6

3.2 Execution and Process Tracing 6

3.3 Analysis and Observations 7

4 PROGRAM EXECUTION ENGINE (MYRUN) **8**

4.1 Implementation of `myrun.c` 8

4.2 Execution and Verification 8

4.3 Analysis and Observations 9

5 I/O REDIRECTION AND PIPELINES **10**

5.1 Implementation of `redirect.c` and `pipeline.c` 10

5.2 Execution and Verification 11

5.3 Analysis and Observations 11

6 BONUS IMPLEMENTATIONS **12**

6.1 Implementation of `myrun2.c` 12

6.2 Implementation of Three-Stage Pipeline (`pipeline3.c`) 12

6.3 Execution and Verification 13

CONCLUSION **14**

REFERENCES **15**

1 INTRODUCTION AND GOAL

1.1 Introduction

A UNIX shell is fundamentally an ordinary userland process that orchestrates command execution through a compact set of kernel system calls: `fork`, `exec`, `wait`, `exit`, `open`, `dup2`, and `pipe`. Rather than being internal kernel constructs, process creation, program replacement, I/O descriptor redirection, and inter-process communication pipelines are composable primitives exposed by the POSIX system-call boundary. Investigating these calls using dynamic system-call tracing tools such as `strace` and implementing the core mechanisms directly in C provides insight into process lifecycle management, kernel process accounting, and user-space I/O buffering.

1.2 Lab Objective

The primary objectives of this laboratory work are:

- inspecting dynamic system call activity generated by standard shells during pipeline construction and directory listing using `strace`;
- analyzing process creation, zombie state persistence, process hierarchy doubling, and standard I/O buffer behavior across forks;
- implementing a custom process runner (`myrun`) to manage program replacement via `execvp` and parse child termination statuses;
- implementing standard file descriptor redirection (`redirect`) using `dup2` to duplicate shell redirection mechanisms;
- implementing unidirectional inter-process pipelines using `pipe` and multi-process execution, examining file descriptor leak hazards and hang conditions;
- extending the core functionality to encompass variable command redirection (`myrun2`) and multi-stage pipelines (`pipeline3`).

2 INSPECTING REAL SHELL SYSTEM CALLS

2.1 System Call Tracing and Execution

To observe how an actual shell constructs pipelines and launches processes, dynamic tracing was executed using `strace`:

```
vitalie@LinuxLabs:~/os-lab7$ strace -f -e trace=pipe2,clone,clone3,dup2,execve,wait4 bash -c 'ls␣/␣|␣wc␣-l'
execve("/usr/bin/bash", ["bash", "-c", "ls␣/␣|␣wc␣-l"], 0x7fffef586d88 /* 55 vars */) = 0
pipe2([3, 4], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7e2ccf8a8a10) = 4851
strace: Process 4851 attached
[pid  4846] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7e2ccf8a8a10) = 4852
strace: Process 4852 attached
[pid  4846] wait4(-1 <unfinished ...>
[pid  4852] dup2(3, 0)                  = 0
[pid  4852] execve("/usr/bin/wc", ["wc", "-l"], 0x5d960a3fd3b0 /* 55 vars */) = 0
[pid  4851] dup2(4, 1)                  = 1
[pid  4851] execve("/usr/bin/ls", ["ls", "/"], 0x5d960a3fd3b0 /* 55 vars */) = 0
[pid  4851] +++ exited with 0 +++
[pid  4846] <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 4851
[pid  4846] wait4(-1 <unfinished ...>
[pid  4852] +++ exited with 0 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 4852
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4851, si_uid=1000, si_status=0, si_utime=0, si_stime=2 /* 0.02 s */} ---
wait4(-1, 0x7ffe0a415994, WNOHANG, NULL) = -1 ECHILD (No child processes)
+++ exited with 0 +++

vitalie@LinuxLabs:~/os-lab7$ strace -c -e trace=write ls /usr/bin > /dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         5           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0         5           total
```

2.2 Analysis and Observations

a) **Order of system calls and descriptor mapping:** The captured trace displays the sequential timeline: `pipe2` → `clone` (PID 4851) → `clone` (PID 4852) → `dup2` → `execve` → `wait4`.

- The left child process (`ls`, PID 4851) calls `dup2(4, 1)`: This duplicates the write end of the newly created pipe (file descriptor 4) onto file descriptor 1 (`STDOUT_FILENO`), directing directory listing data into the pipe buffer.
- The right child process (`wc`, PID 4852) calls `dup2(3, 0)`: This duplicates the read end of the pipe (file descriptor 3) onto file descriptor 0 (`STDIN_FILENO`), causing the word counter utility to ingest stream data directly from the pipe.

b) **Process creation primitive on Linux:** The classic `fork` system call does not appear in the `strace` log. On modern Linux architectures, the GNU C Library (`glibc`) implements `fork()` as an interface

over the more versatile `clone` (or `clone3`) kernel system call, executed with termination signal flag `SIGCHLD`.

c) **Buffering and syscall overhead:** Tracing `ls /usr/bin > /dev/null` revealed that only 5 `write` system calls were performed to output thousands of directory records. When standard output targets a file, pipe, or null device rather than an interactive TTY, the C runtime library transitions from line-buffered mode to fully buffered mode. The output stream buffers blocks in user-space memory and flushes them in aggregated bulk writes, avoiding per-file system-call transition overhead.

3 PROCESS LIFECYCLES, ZOMBIES, AND BUFFERING

3.1 Implementations: `fork.c`, `zombie.c`, and `quiz.c`

The basic process lifecycle, parent-child synchronization, and zombie state retention were tested using three custom utilities:

Listing 3.1: Source code of `fork.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("before␣fork:␣pid=%d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        printf("child:␣pid=%d␣parent=%d\n", getpid(), getppid());
        exit(42);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent:␣pid=%d,␣my␣child␣%d␣exited␣with␣%d\n",
           getpid(), pid, WEXITSTATUS(status));
    return 0;
}
```

In `zombie.c`, an intentional synchronization delay was inserted by calling `sleep(30)` in the parent immediately prior to `waitpid`. In `quiz.c`, three sequential `fork()` invocations were placed prior to a single `printf("hi\n")`.

3.2 Execution and Process Tracing

```
vitalie@LinuxLabs:~/os-lab7$ ./fork
before fork: pid=4887
child: pid=4889 parent=4887
parent: pid=4887, my child 4889 exited with 42

vitalie@LinuxLabs:~/os-lab7$ ./zombie &
[1] 4919
before fork: pid=4919
child: pid=4922 parent=4919

vitalie@LinuxLabs:~/os-lab7$ sleep 1; ps -o pid,ppid,stat,cmd --ppid $!
    PID    PPID STAT CMD
   4922    4919 Z    [zombie] <defunct>

vitalie@LinuxLabs:~/os-lab7$ ./quiz | wc -l
```

```
8

vitalie@LinuxLabs:~/os-lab7$ ./fork
before fork: pid=4971
child: pid=4972 parent=4971
parent: pid=4971, my child 4972 exited with 42

vitalie@LinuxLabs:~/os-lab7$ ./fork | cat
before fork: pid=4973
child: pid=4975 parent=4973
before fork: pid=4973
parent: pid=4973, my child 4975 exited with 42
```

3.3 Analysis and Observations

a) **Zombie process state and reclamation:** While the parent process was suspended inside `sleep(30)`, the child process had already completed its execution via `exit(42)`. In the process table, the child's status registered as **Z** with command descriptor `[zombie] <defunct>`. The kernel preserves this minimal entry in the process table to hold the termination status until harvested. As soon as the parent awoke and invoked `waitpid`, the exit status 42 was collected and the zombie was purged from the kernel table.

b) **Process tree exponential doubling:** Running `quiz | wc -l` returned exactly 8 output lines. Each `fork()` duplicates all active processes, yielding $2^3 = 8$ concurrent processes:

$$N_{\text{processes}} = 2^k = 2^3 = 8 \tag{3.1}$$

where k is the number of sequential `fork()` invocations.

c) **Standard I/O buffer duplication puzzle:** When `fflush(stdout)` was omitted, running `./fork` in a terminal displayed "`before fork`" once because interactive standard output is line-buffered; the terminating newline flushed the buffer immediately. However, piping into `cat` converted `stdout` into fully buffered mode. The initial string was retained in user-space buffer memory. When `fork()` was subsequently called, the kernel copied the parent's entire address space—including the unflushed buffer—into the child. Upon termination, both parent and child flushed their respective copies, causing the string to be printed twice.

4 PROGRAM EXECUTION ENGINE (MYRUN)

4.1 Implementation of `myrun.c`

The `myrun` utility duplicates the process execution cycle of a shell, replacing child process memory via `execvp` and evaluating exit status macros:

Listing 4.1: Source code of `myrun.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s program [args...]\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror("execvp");
        exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("[myrun] %s exited with status %d\n", argv[1], WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("[myrun] %s killed by signal %d\n", argv[1], WTERMSIG(status));
    }

    return 0;
}
```

4.2 Execution and Verification

All four operational outcomes were validated:

```
vitalie@LinuxLabs:~/os-lab7$ ./myrun ls -d /etc
/etc
[myrun] ls exited with status 0

vitalie@LinuxLabs:~/os-lab7$ ./myrun false
[myrun] false exited with status 1

vitalie@LinuxLabs:~/os-lab7$ ./myrun nosuchprog
```

```
execvp: No such file or directory
[myrun] nosuchprog exited with status 127

vitalie@LinuxLabs:~/os-lab7$ ./myrun sleep 100 & sleep 1; pkill -x sleep
[1] 5024
[myrun] sleep killed by signal 15
[1]+  Done                    ./myrun sleep 100
```

4.3 Analysis and Observations

a) **Program replacement semantics:** Code following `execvp` only runs if the system call fails. Upon a successful call to `execvp`, the kernel replaces the existing code segment, data, stack, and heap with the target binary. The original execution path ceases to exist in memory.

b) **Exit status 127 convention:** Exit status 127 is the standard POSIX convention designating command lookup failure (`command not found`). Executing an invalid command in Bash (`bash -c nosuchprog; echo $?`) yields 127 identically.

c) **Signal termination:** The test terminating `sleep` reported termination by signal **15**, which corresponds to `SIGTERM`, the default termination signal delivered by `pkill`.

5 I/O REDIRECTION AND PIPELINES

5.1 Implementation of `redirect.c` and `pipeline.c`

File descriptor manipulation was implemented to redirect stream output and connect process pipelines:

Listing 5.1: Descriptor redirection in `redirect.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }

    if (pid == 0) {
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }

        if (dup2(fd, STDOUT_FILENO) < 0) {
            perror("dup2");
            exit(1);
        }
        close(fd);

        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    return 0;
}
```

Listing 5.2: Pipeline orchestration in `pipeline.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); exit(1); }

    pid_t pid1 = fork();
    if (pid1 == 0) {
        dup2(p[1], STDOUT_FILENO);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "-l", "/etc", (char *)NULL);
        perror("execlp␣ls");
        exit(127);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
```

```
        dup2(p[0], STDIN_FILENO);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp wc");
        exit(127);
    }

    close(p[0]);
    close(p[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
    return 0;
}
```

5.2 Execution and Verification

Output validation and pipeline comparison against native bash utilities:

```
vitalie@LinuxLabs:~/os-lab7$ ./redirect && cat out.txt | head -3
total 104
-rwxrwxr-x 1 vitalie vitalie 16344 Oct 16 07:29 fork
-rw-rw-r-- 1 vitalie vitalie   575 Oct 16 07:29 fork.c

vitalie@LinuxLabs:~/os-lab7$ ./pipeline
238
vitalie@LinuxLabs:~/os-lab7$ ls -l /etc | wc -l
238
```

5.3 Analysis and Observations

a) **Pipe closure requirements and hang state:** When `close(p[1])` was intentionally commented out in the parent process, executing `./pipeline` hung indefinitely until terminated with `Ctrl-C`. The downstream process (`wc`) consumes data from `p[0]` until reaching an end-of-file condition (`EOF`). The operating system kernel only raises `EOF` on a pipe when every open descriptor referencing its write end across all processes is closed. Because the parent retained an open reference to `p[1]`, `wc` remained blocked on `read()`.

b) **Child vs. Parent redirection isolation:** Descriptor duplication via `dup2` must occur in the child process between `fork` and `exec`. If the shell modified its own descriptor table directly, it would overwrite its standard input and output streams, severing terminal interaction and preventing further command execution.

6 BONUS IMPLEMENTATIONS

6.1 Implementation of `myrun2.c`

The `myrun2` utility merges dynamic command execution with arbitrary file redirection:

Listing 6.1: Source code of `myrun2.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s <output_file> <program> [args...]\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }

        dup2(fd, STDOUT_FILENO);
        close(fd);

        execvp(argv[2], &argv[2]);
        perror("execvp");
        exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }

    if (WIFEXITED(status)) {
        printf("[myrun2] %s exited with status %d\n", argv[2], WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("[myrun2] %s killed by signal %d\n", argv[2], WTERMSIG(status));
    }
    return 0;
}
```

6.2 Implementation of Three-Stage Pipeline (`pipeline3.c`)

To execute `ls /usr/bin | grep x | wc -l`, two distinct pipes and three concurrent children were created:

Listing 6.2: Source code of `pipeline3.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```c
int main(void)
{
    int p1[2], p2[2];
    if (pipe(p1) < 0 || pipe(p2) < 0) { perror("pipe"); exit(1); }

    /* Child 1: ls /usr/bin */
    pid_t pid1 = fork();
    if (pid1 == 0) {
        dup2(p1[1], STDOUT_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("execlp␣ls");
        exit(127);
    }

    /* Child 2: grep x */
    pid_t pid2 = fork();
    if (pid2 == 0) {
        dup2(p1[0], STDIN_FILENO);
        dup2(p2[1], STDOUT_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("execlp␣grep");
        exit(127);
    }

    /* Child 3: wc -l */
    pid_t pid3 = fork();
    if (pid3 == 0) {
        dup2(p2[0], STDIN_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp␣wc");
        exit(127);
    }

    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
    waitpid(pid3, NULL, 0);
    return 0;
}
```

6.3 Execution and Verification

```
vitalie@LinuxLabs:~/os-lab7$ ./myrun2 out2.txt ls -l /etc && head -3 out2.txt
[myrun2] ls exited with status 0
total 1160
drwxr-xr-x  5 root root    4096 Aug 26 05:30 ModemManager
drwxr-xr-x  8 root root    4096 Oct  4 18:57 NetworkManager

vitalie@LinuxLabs:~/os-lab7$ ./pipeline3
233
vitalie@LinuxLabs:~/os-lab7$ ls /usr/bin | grep x | wc -l
233
```

CONCLUSION

This laboratory work investigated the fundamental mechanisms underlying POSIX process orchestration and inter-process communication in Linux. By combining dynamic system call tracing with concrete C implementations, the behavior of shell building blocks was systematically examined.

Key conclusions drawn from this laboratory include:

- **System Call Orchestration:** Standard shells rely on simple primitive operations (`clone`, `dup2`, `execve`, `wait4`, and `pipe2`) rather than specialized monolithic kernel routines.
- **Process Lifecycles and Accounting:** Children that exit prior to their parent's collection persist in a zombie (`Z`) state, retaining their exit status until purged via `waitpid`.
- **I/O Buffering Mechanics:** Standard I/O behavior differs substantially depending on destination streams. Fully buffered streams duplicated during `fork` cause repeated outputs unless explicitly flushed via `fflush`.
- **Descriptor Hygiene in Pipelines:** Robust inter-process pipelines require diligent closing of unused pipe descriptors across both parents and children; leaving write ends open prevents EOF transmission and induces deadlocks.

Overall, completing the core tasks along with the bonus implementations (`myrun2` and `pipeline3`) provided a practical grasp of operating system boundaries and concurrent program execution.

REFERENCES

[1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018.

[2] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[3] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, Addison-Wesley Professional, 3rd edition, 2013.

[4] Linux Programmer's Manual, *fork(2), execve(2), waitpid(2), pipe(2), dup(2) Man Pages*.